

COMP6006 Final Assignment (Part A)

Conference Connect: A Real-Time Conference Engagement Platform

Assignment Description

Total Marks: 50 **Due Date: 23:59 1st June**

Overview

In this assignment, you will develop **Conference Connect**, a real-time conference engagement platform designed for professional events and technology conferences. Users will be able to log in using **LinkedIn authentication**, browse through available conference sessions, save sessions to their personal schedule, and join live session rooms where they can interact with other attendees in real time. The application must be built as a **React-based Single Page Application (SPA)** utilising **React Router** for navigation, **React-Bootstrap** for responsive user interface design, and **Socket.io** for real-time communication between users.

This assignment focuses on:

- React.js frontend development
- SPA navigation and routing
- Third-party authentication
- Real-time communication using Socket.io
- Component-based UI design
- Frontend state management

Note: A database is **not required** for this assignment. In-memory storage on the server side may be used where appropriate.

Assignment Objectives

By completing this assignment, students will demonstrate their ability to:

- Develop a React-based SPA using reusable components
- Implement page navigation using React Router
- Implement LinkedIn OAuth authentication using Passport.js
- Develop real-time applications using Socket.io
- Manage frontend state and dynamic UI updates
- Design responsive interfaces using React-Bootstrap
- Build protected routes and authenticated functionality

Scenario

A technology conference organiser requires a web platform where attendees can:

- Authenticate using LinkedIn
- Browse conference sessions
- Save sessions to a personal schedule
- Join live conference session rooms
- Communicate with other attendees in real time

The platform should support multiple live sessions simultaneously.

Each live session room should maintain:

- Active participant lists
- Real-time chat
- Chat history while the server is running

Functional Requirements

1. React Frontend with React-Bootstrap (25 Marks)

Develop the frontend as a React Single Page Application (SPA).

Frontend Requirements

Setup

- Use either:
- Vite, or
- Create React App
- Use React-Bootstrap for interface design
- The application must implement protected routes to prevent unauthenticated users from accessing conference pages and live session rooms.
- The application must use React Router for SPA navigation between pages.

Login Page

Users must be able to:

- Log in using LinkedIn OAuth
- Log out securely
- View appropriate error messages if authentication fails

After successful login:

- Redirect users to the dashboard page

Important Note: If you are unable to implement LinkedIn OAuth authentication, you may alternatively use the Passport Local Strategy similar to the implementation used in the practical workshops. However, no marks will be awarded for the authentication section if LinkedIn OAuth is not implemented. Using local authentication will still allow you to implement protected routes and authenticated functionality required in other sections of the assignment. Applications without any login or authentication implementation may lose marks across multiple sections where authenticated access and route protection are expected.

Dashboard Page

The dashboard should display all available conference sessions using a responsive card-based layout. Each session should present:

- Session title
- Speaker name
- Topic/category
- Scheduled time
- Short description
- Join Live Session button
- Add to My Schedule button

Students are expected to use appropriate React-Bootstrap components to design the interface, such as:

- Card
- Button
- Badge
- Container
- Row
- Col
- Navbar

The dashboard layout should be responsive and organised appropriately for different screen sizes.

My Schedule Page

Users should be able to:

- View sessions added to their personal schedule
- Remove sessions from their schedule
- Directly join a live session from the schedule page

The interface should update dynamically without requiring a full page refresh. Saved conference sessions should be displayed using reusable UI components consistent with the dashboard design.

Navigation Bar

The navigation bar should include:

- Conference platform name/logo
- Logged-in username
- Profile image if available
- Navigation links
- Logout button

Navigation links should include:

- Dashboard
- My Schedule
- Active Sessions

Active Sessions Page

Display currently active live conference rooms. A session room should be considered active when at least one authenticated participant has joined the room. Each room should display:

- Session title
- Current participant count
- Button to join the room

The participant count should update automatically in real time for all connected users when participants join or leave a session room, without requiring a page refresh.

2. LinkedIn Authentication using Passport.js (5 Marks)

Implement LinkedIn authentication using Passport.js.

Requirements

- Use:
- passport
- passport-linkedin-oauth2
- Store LinkedIn credentials securely using environment variables
- After successful login:
- Create an authenticated session
- Redirect the user to the dashboard

The application should display:

- Attendee name
- LinkedIn profile image if available

3. Real-Time Conference Session System using Socket.io (20 Marks)

Implement real-time communication features using Socket.io.

Live Session Rooms

Users should be able to join live conference rooms. Each live room must support:

- Multiple simultaneous users
- Real-time communication
- Participant tracking
- Users should be able to leave one live session room and join another session room during runtime.

Socket.io rooms should be used to ensure messages are only delivered to users within the corresponding conference session.

Live Chat

Users in a session room should be able to:

- Send messages
- Receive messages instantly
- View chat history while the server remains active. Permanent database storage is not required.

Each message should display:

- Sender name
- Timestamp
- Message content

Active Participant List

Each session room should display:

- Currently connected participants
- Participant updates when users join or leave

Participant lists should update automatically in real time.

Join and Leave Notifications

Users should receive live notifications when:

- A participant joins the room
- A participant leaves the room

Technical Requirements

Frontend

- React.js
- React Router
- React-Bootstrap
- The application interface should be responsive and usable across different screen sizes.

Backend

- Node.js
- Express.js

Authentication

- Passport.js
- LinkedIn OAuth

Real-Time Communication

- Socket.io

Security Requirements

Only authenticated users should be able to:

- Access protected pages
- Join live session rooms
- Send chat messages
- Manage personal schedules

Important Notes

- A database is not required.
- In-memory server-side storage may be used. Students should understand that all in-memory data will reset when the server restarts.
- Conference session data may be hardcoded.
- You should provide a reasonable set of pre-defined conference sessions for demonstrating application functionality.
- The application must function correctly while the server is running.
- You may structure your frontend and backend folders as you prefer.

Suggested Routes

Example routes may include:

- /login
- /dashboard
- /my-schedule
- /active-sessions

- /live/:sessionId

You may implement additional routes if required.

Submission Requirements

Submit a single .zip file containing:

- Frontend source code
- Backend source code

Do not include:

- **node_modules/**

Your submission must include:

- Setup instructions
- Commands required to run the project
- Environment variable instructions
- **Do not include actual LinkedIn credentials or secrets in your submission.**

Demonstration Requirements

Students must demonstrate their application during the scheduled demonstration session.

During the demonstration, students may be asked to:

- Explain application architecture
- Explain Socket.io implementation
- Explain authentication flow
- Explain React component structure
- Explain routing/navigation logic
- **Students should be prepared to modify or explain selected sections of code during the demonstration.**

Failure to adequately explain the submitted work may affect marks.

Academic Integrity

All submitted work must be your own.

Students must be able to explain:

- Implemented features
- Frontend logic
- Backend logic
- Real-time communication functionality

Use of external tutorials or resources without understanding the implementation may affect assessment outcomes.